



Prisoner's Dilemma Strategy Simulator

Foundations of AI (FAI) — In-Session Activity



Objective

Build a simple interactive **Prisoner's Dilemma Strategy Simulator** using:

- Basic Python
- Simple conditional logic
- Lists and loops
- Gradio GUI
- Google Colab

Students will:

- Understand cooperation vs betrayal
 - Simulate game theory decisions
 - Compare scores between players
 - Build a working AI simulation app
-



Concepts Covered

This activity reinforces:

- Game Theory
 - Strategic reasoning
 - Decision making
 - Simulation systems
 - Conditional statements
 - Loops
 - Lists
 - Interactive AI applications
-



Suggested Time Breakdown (120 Minutes)

Task	Time
Understand Prisoner's Dilemma	15 mins
Create payoff logic	20 mins
Implement simulation	25 mins
Build Gradio GUI	30 mins
Testing & improvements	30 mins



Understanding the Prisoner's Dilemma

Two players independently choose:

- Cooperate
- Betray

Each combination gives different rewards.



Example Payoff Matrix

Player A	Player B	A Score	B Score
Cooperate	Cooperate	3	3
Betray	Cooperate	5	0
Cooperate	Betray	0	5
Betray	Betray	1	1



Step-by-Step Implementation

◆ Step 1: Open Google Colab

Open:

- Google Colab
 - Create a new notebook
-

◆ Step 2: Install Gradio

Run this cell:

```
!pip install gradio
```

◆ Step 3: Import Libraries

```
import random
import gradio as gr
```

◆ Step 4: Create Payoff Logic

Purpose

Calculate scores based on player choices.

```
def get_scores(move1, move2):

    if move1 == "Cooperate" and move2 == "Cooperate":
        return 3, 3

    elif move1 == "Betray" and move2 == "Cooperate":
        return 5, 0

    elif move1 == "Cooperate" and move2 == "Betray":
        return 0, 5
```

```
else:
    return 1, 1
```

◆ Step 5: Create Simple AI Strategies

Purpose

Create different behaviors.

```
def ai_move(strategy):

    if strategy == "Always Cooperate":
        return "Cooperate"

    elif strategy == "Always Betray":
        return "Betray"

    else:
        return random.choice(["Cooperate", "Betray"])
```

◆ Step 6: Create Main Simulation Function

Purpose

Run multiple rounds and calculate scores.

```
def play_game(player1_strategy, player2_strategy, rounds):

    score1 = 0
    score2 = 0

    result = ""

    for i in range(rounds):

        move1 = ai_move(player1_strategy)
        move2 = ai_move(player2_strategy)

        points1, points2 = get_scores(move1, move2)
```

```

        score1 += points1
        score2 += points2

        result += f"Round {i+1}"
    "
    result += f"Player A: {move1}"
    "
    result += f"Player B: {move2}"
    "
    result += f"Scores: {score1} - {score2}"
    "

    result += "Final Scores"
    "
    result += f"Player A Total: {score1}"
    "
    result += f"Player B Total: {score2}"
    "

    return result

```

◆ Step 7: Create Gradio Interface

Purpose

Build interactive GUI application.

```

app = gr.Interface(
    fn=play_game,

    inputs=[
        gr.Dropdown(
            ["Always Cooperate", "Always Betray", "Random"],
            label="Player A Strategy"
        ),

        gr.Dropdown(
            ["Always Cooperate", "Always Betray", "Random"],
            label="Player B Strategy"
        ),
    ],

```

```
        gr.Slider(1, 20, value=5, label="Rounds")
    ],

    outputs="text",

    title="Prisoner's Dilemma Strategy Simulator",

    description="Compare different game theory strategies"
)
```

◆ Step 8: Launch Application

```
app.launch()
```

◆ Step 9: Run and Test

Students should:

- Select strategies
- Choose rounds
- Run simulation
- Compare scores

Expected Output

Students should see:

- Dropdowns for strategies
- Round selection slider
- Round-by-round results
- Final scores

Example:

```
Round 1
Player A: Cooperate
```

Player B: Betray
Scores: 0 - 5

Bonus Features

1. Add Human vs AI Mode

Allow user to manually select:

- Cooperate
- Betray

against AI.

2. Add Score Charts

Use:

- matplotlib

Plot:

- score progression
 - cooperation frequency
-

3. Add New Strategy

Example:

- Tit-for-Tat
 - Mostly Cooperate
 - Mostly Betray
-

4. Add Winner Display

Show:

```
if score1 > score2:  
    winner = "Player A Wins"
```



Discussion Questions

1. Which strategy scored highest?
2. Is cooperation always beneficial?
3. What happens when both betray?
4. Which strategy is safest?
5. Where is this seen in real life?



Real-World Applications

This concept is used in:

- Business competition
- Pricing strategies
- Cybersecurity
- International relations
- AI agents
- Negotiation systems



Common Requirements

Students must:

- Use simple Python
- Build Gradio interface
- Simulate minimum 5 rounds
- Display scores clearly
- Use loops and conditions properly



Submission Requirements

Submit:

- Google Colab notebook link

- Screenshot of Gradio application
 - Short explanation of strategies used
-



Final Goal

Build a beginner-friendly game theory simulator that demonstrates cooperation, betrayal, and strategic decision making using Python and Gradio.